

Project 2: Image Classification Using Convolutional Neural Networks (CNN)

Course Context

Month 5 — Week 2

Deep Learning Architectures: ANN and CNN

1. Problem Statement

1.1 Background

Image classification is a core task in computer vision. Companies use it for product tagging, quality inspection, and content moderation. In this project, you will build a **Convolutional Neural Network (CNN)** to classify images into predefined categories using a standard benchmark dataset.

1.2 Objective

Build a **CNN** using TensorFlow/Keras that:

- Takes grayscale or RGB images as input.
- Classifies each image into one of several classes (e.g., clothing categories in Fashion-MNIST, or binary classes such as cats vs dogs if using a different dataset).
- Achieves reasonable accuracy on a held-out test set and uses basic regularization and data augmentation to improve generalization.

1.3 Technical Scope

- Load and preprocess image data (normalization, resizing if required, train/validation/test split).
- Design a CNN with:
- Convolutional layers with appropriate filter sizes and pooling.
- Flattening and dense layers for classification.
- Suitable activation functions (e.g., ReLU in conv/dense, softmax at output for multi-class).
- Apply regularization (e.g., dropout, L2) and/or early stopping to reduce overfitting.
- Optionally apply data augmentation (e.g., rotation, flip, zoom) to improve robustness.
- Evaluate using accuracy and confusion matrix; optionally visualize filters or feature maps.

1.4 Success Criteria

- A working CNN that trains and evaluates without errors.
- Test accuracy and confusion matrix reported.

- Use of at least one regularization technique and one data augmentation (or clear comparison with/without).

1.5 Constraints and Assumptions

- Dataset: Fashion-MNIST (or instructor-provided alternative) is recommended for a standardized, downloadable dataset.
- Code must be reproducible (fixed random seed).
- Images should be normalized (e.g., pixel values in $[0, 1]$).

2. Expected Deliverables

1. Code: Jupyter notebook or Python script: data load, preprocessing, CNN build, training, evaluation.
2. Output: Test accuracy, confusion matrix, and optional training curves or filter visualizations.
3. Short summary: Architecture description, effect of dropout/augmentation on validation performance.

3. Learning Outcomes Addressed

- Understanding CNN structure: convolutional layers, pooling, flattening, dense layers.
- Building and training a CNN for image classification in Keras.
- Using dropout, early stopping, and data augmentation to improve generalization.